Optimización de Consultas e Índices — Apunte de estudio

Bases de Datos 2 · Ingeniería de Sistemas — Sep 21, 2026

Apunte armado para usarse como contexto en una conversación con una IA: adjuntalo entero al chat junto con el prompt sugerido del final.

1. Cómo piensa el optimizador

Cuando mandás un SELECT, el motor no lo ejecuta tal cual lo escribiste. Lo procesa en 4 pasos: **entender** (arma una primera representación interna, un árbol de consulta), **optimizar** (prueba formas equivalentes de ese árbol y estima cuál es más barata), **generar** (traduce la estrategia elegida a código de bajo nivel) y **ejecutar** (corre ese código sobre los datos reales).

Hay dos objetivos que conviene no confundir: *procesar* la consulta es transformar el SQL en UNA estrategia válida (que dé el resultado correcto); *optimizar* es, entre TODAS las estrategias válidas posibles, elegir la que gasta menos recursos. Analizar todas las estrategias posibles para encontrar la óptima sería carísimo — el optimizador no busca la perfecta, busca una muy buena, y lo logra casi siempre.

Para eso combina dos técnicas. La **heurística** reordena las operaciones de la consulta usando reglas que casi siempre mejoran el rendimiento, sin necesidad de calcular costos exactos. La **estimación de costos** calcula un costo aproximado (uso de disco, memoria, tiempo) para varias estrategias y elige la de menor costo estimado. Primero se aplica la heurística para reducir las alternativas; después se comparan costos entre las que quedan.

"Estimar" un costo quiere decir tres cosas: el optimizador no mide (no ejecuta nada todavía), calcula un número que representa cuánto trabajo llevaría un plan — es una proyección, no el tiempo real; compara ese número entre varios planes candidatos (distinto orden de joins, distintos algoritmos); y se queda con el de menor costo estimado, no con el "perfecto".

¿De dónde saca esos números? De estadísticas que guarda sobre cada tabla. A nivel de tabla: cardinalidad (cantidad total de filas), factor de bloqueo (cuántas filas entran en cada bloque de disco), bloques ocupados, y qué índices existen. A nivel de columna: valores distintos (NDV), valores mínimo y máximo, y la **selectividad**.

La selectividad mide qué tan bien filtra una columna, y se calcula como:

```
\text{selectividad} = \frac{\text{valores distintos}}{\text{cardinalidad}}
```

Cuanto más cerca de 1, mejor filtra (cada valor aparece pocas veces). Cuanto más cerca de 0, peor filtra (cada valor se repite mucho). Ejemplo: PROYECTO tiene 200 filas y 200 valores distintos de `nombre` → selectividad = 200/200 = 1 (altísima: filtrar por un nombre puntual deja ~1 fila). EMPLEADO tiene 5.000 filas pero `cargo` toma solo un puñado de valores distintos, digamos 8 → selectividad = 8/5.000 = 0,0016 (bajísima: filtrar por un cargo puntual sigue dejando ~625 filas). Si estas estadísticas quedan desactualizadas, el optimizador puede terminar eligiendo un plan que ya no conviene — por eso en Postgres conviene correr `ANALYZE` después de cambios grandes en los datos.

2. Álgebra relacional: el lenguaje de las reescrituras

El álgebra relacional es la forma en que el optimizador representa tu consulta por dentro: traduce el SQL a una expresión armada como un árbol (el árbol de consulta). No es matemática abstracta y lejana — el plan que te muestra EXPLAIN es, en el fondo, ese mismo árbol con nombres físicos (Seq Scan, Hash Join, etc.).

Tres operadores básicos:

Símbolo	Nombre	Equivale a
σ (sigma)	Filtrar filas	tu cláusula WHERE
π (pi)	Elegir columnas	tu cláusula SELECT
⋈ (join)	Cruzar tablas	tu FROM + la condición de unión

(También existen ∪, ∩ y − — las operaciones clásicas de conjuntos, aplicadas a tablas.)

Con esos operadores definidos, se pueden escribir con precisión las transformaciones que aplica el optimizador. La bibliografía documenta unas 12 reglas de equivalencia formales; se resumen en **3 reglas de oro**:

1. Filtrá lo antes posible. Si una condición (σ) solo necesita datos de UNA tabla, aplicala apenas leas esa tabla, sin esperar a cruzarla con las demás.

```
\sigma_c(A \Join B) \equiv (\sigma_c(A)) \Join B \quad \text{(válido cuando \(c\) solo usa columnas de \(A\))}
```

2. Llevá solo las columnas que necesitás. Si al final vas a descartar columnas, descartalas lo antes posible: cada cruce que sigue en la cadena trabaja con menos datos.

```
\pi_L(A \Join B) \equiv \pi_L(\pi_{L1}(A) \Join \pi_{L2}(B))
```

3. El orden de los cruces se puede acomodar. Los join, uniones e intersecciones no cambian de resultado si cambiás el orden de las tablas ni la forma en que las agrupás (conmutatividad y asociatividad). Esta libertad es justo la que le permite al optimizador probar varios órdenes y quedarse con el más barato.

```
A \Join B \equiv B \Join A \qquad (A \Join B) \Join C \equiv A \Join (B \Join C)
```

En resumen, frente a una consulta real el optimizador intenta: aplicar los filtros lo antes posible; empezar por el filtro más restrictivo (el que menos filas deja); convertir “cruzar todo y filtrar después” en un join directo; y descartar columnas que no hacen falta, apenas puede.

3. IN, EXISTS y NULLs: otra reescritura del optimizador

Escribir un IN o un EXISTS con subconsulta no significa que el motor la ejecute fila por fila, como sugiere la sintaxis. Casi siempre el optimizador lo reescribe como un cruce especial: el **semi-join** (⋉). A ⋉ B devuelve las filas de A que tienen al menos una coincidencia en B, pero solo trae columnas de A — nunca las de B — y nunca duplica una fila de A aunque matchee con varias de B. Esa es la diferencia clave con un ⋈ normal, que combina columnas de ambas tablas y puede duplicar filas cuando hay varias coincidencias.

Ejemplo: `E.id_empleado IN (SELECT T.id_empleado FROM TRABAJA_EN T WHERE T.cod_proyecto = 'ROBOTICS')` equivale a `EMPLEADO ⋉ TRABAJA_EN`. Un EXISTS con subconsulta correlacionada se resuelve igual: el optimizador la "decorrelaciona" y la ejecuta, en la práctica, como ese mismo semi-join.

En el plan de ejecución esto queda explícito: Postgres nombra el nodo "Hash Semi Join" o "Nested Loop Semi Join" (según el algoritmo elegido), y para el caso NOT EXISTS / NOT IN seguro, "Anti Join".

El caso traicionero: NOT IN con NULLs. Acá hay que separar dos situaciones:

- Si el NULL está en la subconsulta (la lista de valores), **IN funciona perfecto** — el que se rompe es NOT IN. `x IN (v1, v2, NULL)`: si x coincide con algún valor, da TRUE (correcto); si no coincide con nada, da UNKNOWN en vez de FALSE, pero como WHERE descarta tanto FALSE como UNKNOWN, el resultado final no cambia.
- `x NOT IN (v1, v2, NULL)` equivale a `x<>v1 AND x<>v2 AND x<>NULL`. Si x coincide con algún valor, da FALSE (correcto, queda afuera). Si x no coincide con nada, todas las comparaciones contra los valores reales dan TRUE, pero `x<>NULL` da UNKNOWN — y TRUE AND ... AND UNKNOWN = UNKNOWN, así que la fila queda afuera igual, aunque en realidad x no está en la lista. Como esto pasa para toda fila que no matchea, la consulta entera puede devolver **cero filas, sin ningún error que avise**.

NOT EXISTS no tiene este problema: compara existencia de filas, no valores puntuales, así que nunca lo afecta un NULL en la subconsulta — siempre se puede convertir en un anti-join seguro. Regla práctica: si no podés garantizar que la columna de la subconsulta sea NOT NULL, preferí NOT EXISTS.

(Aparte: si el NULL está del lado de la columna externa — x es NULL — tanto IN como NOT IN excluyen esa fila. Pero eso es el comportamiento normal de NULL en cualquier comparación, igual que `x = 5`, no un defecto específico de IN/NOT IN.)

4. Ejercicio guiado, resuelto paso a paso

El caso: queremos el apellido de los empleados que trabajan en el proyecto ROBOTICS y tienen cargo de investigador. Modelo: `EMPLEADO(id_empleado PK, nombre, apellido, cargo)` — 1:N — `TRABAJA_EN(id_empleado PK/FK, cod_proyecto PK/FK, cant_horas)` — N:1 — `PROYECTO(cod_proyecto PK, nombre)`.

```
SELECT E.apellido
FROM   EMPLEADO E, TRABAJA_EN T, PROYECTO P
WHERE  E.id_empleado = T.id_empleado
AND    P.cod_proyecto = T.cod_proyecto
AND    P.nombre = 'ROBOTICS'
AND    E.cargo = 'investigador';
```

Paso 1 — traducción directa a álgebra (sin optimizar todavía):

```
\pi_{\text{apellido}}\big(\sigma_{\,4\ \text{condiciones}}(\text{EMPLEADO} \times \text{TRABAJA\_EN} \times \text{PROYECTO})\big)
```

Este árbol inicial todavía usa el producto cartesiano (×), el operador más costoso de todos: cruza las 3 tablas completas antes de aplicar cualquier filtro.

Paso 2 — aplicamos las 3 reglas de oro:

1. Separamos el filtro grande (4 condiciones juntas) en 4 filtros chicos, para que cada uno pueda viajar por separado.
2. Empujamos cada filtro junto a SU tabla (regla 1): `P.nombre='ROBOTICS'` va sobre PROYECTO, `E.cargo='investigador'` va sobre EMPLEADO. Las condiciones de igualdad entre tablas pasan a ser la condición del join.
3. Elegimos qué cruzar primero (regla 3 + heurística): PROYECTO filtrado por ROBOTICS deja 1 sola fila, así que conviene cruzarlo primero con TRABAJA_EN, y recién al final incorporar EMPLEADO.

Árbol optimizado (mismo resultado exacto, ejecución mucho más eficiente):

```
π apellido
  |
  +- ⋈ (id_empleado)
       |
       +- σ cargo='investigador'
       |    |
       |    +- EMPLEADO
       |
       +- ⋈ (cod_proyecto)
            |
            +- σ nombre='ROBOTICS'
            |    |
            |    +- PROYECTO
            |
            +- TRABAJA_EN
```

¿Cómo sabe el optimizador que ya encontró un buen plan? Repite un ciclo: (1) aplica una transformación válida sobre el árbol actual y genera un plan candidato; (2) estima su costo usando las estadísticas; (3) compara — ¿es más barato que el mejor plan encontrado hasta ahora? si sí, lo reemplaza; (4) si todavía quedan transformaciones razonables por probar, vuelve al paso 1. Se detiene cuando ya no hay una transformación que mejore el costo, o cuando seguir buscando costaría más de lo que se ahorraría.

5. Leer planes reales: EXPLAIN y EXPLAIN ANALYZE

Algunos SGBD permiten inspeccionar la estrategia elegida por el optimizador. `EXPLAIN` te muestra el plan real que eligió — no una suposición —: qué operación usa en cada tabla (recorrido completo o por índice) y qué algoritmo eligió para cada cruce. `EXPLAIN ANALYZE` además ejecuta la consulta de verdad y te dice cuánto tardó cada paso (tiempo real, no solo estimado). En Postgres, `EXPLAIN` te muestra el plan elegido pero no te deja fijarlo manualmente.

Cómo leerlo, simple: empezá por la línea más indentada (es lo que se ejecuta primero); cada resultado "sube" y alimenta a la línea de arriba; buscá el nodo con mayor costo o tiempo real — ahí está el cuello de botella.

El resultado real de nuestro ejercicio. Corriendo `EXPLAIN ANALYZE` sobre la consulta de la sección anterior, Postgres devuelve:

```
Nested Loop  (cost=8.45..16.62 rows=1) (actual rows=1 loops=1)
  ->  Nested Loop  (cost=8.16..15.41 rows=4) (actual rows=4 loops=1)
        ->  Index Scan using idx_proyecto_nombre on proyecto p
              (cost=0.28..8.30)  Index Cond: (nombre = 'ROBOTICS')
```

```
       ->  Index Scan using idx_trabaja_en_proyecto on trabaja_en t
             (cost=0.29..7.06)  Index Cond: (cod_proyecto = p.cod_proyecto)
  ->  Index Scan using empleado_pkey on empleado e
       (cost=0.29..0.31)  Index Cond: (id_empleado = t.id_empleado)
       Filter: (cargo = 'investigador'::text)

Planning Time: 0.312 ms      Execution Time: 0.061 ms
```

Los tres accesos son `Index Scan` — ni un `Seq Scan` — y arranca por PROYECTO filtrado por `'ROBOTICS'`, el paso más selectivo. Es, en la práctica, el mismo plan optimizado que armamos a mano en la sección 4: se resuelve en 0,061 ms.

Herramientas para leer un plan sin sufrir: tatiyants.com/pev pega la salida de EXPLAIN y la convierte en un diagrama interactivo, resaltando los pasos más costosos; explain.dalibo.com hace lo mismo, con foco en comparar tiempo estimado vs. tiempo real de cada nodo.

6. Índices: qué son y cómo funcionan

Como el índice de un libro: para encontrar "optimización" en un libro de 400 páginas no leés página por página — vas al índice del final, que te dice en qué página está, y saltás directo ahí. En una base de datos, un **índice** es una estructura de datos auxiliar que el motor construye y mantiene ordenada por los valores de una o más columnas, para encontrar filas sin recorrer la tabla completa. No es magia: es otra estructura, separada de la tabla, que ocupa su propio espacio en disco.

Por cada fila, guarda el valor de la columna indexada junto con un puntero a dónde está esa fila completa. Buscar en esa estructura chica y ordenada es mucho más barato que recorrer la tabla entera fila por fila. Sirve para acelerar búsquedas (WHERE), cruces (JOIN) y ordenamientos (ORDER BY) — y para garantizar unicidad, como en las claves primarias.

Sin índice → `Seq Scan`: el motor recorre la tabla entera hasta encontrar las filas que cumplen la condición. Con índice → `Index Scan`: el motor salta directo a esas filas, sin tocar el resto de la tabla.

Sintaxis básica en PostgreSQL:

```
CREATE INDEX idx_nombre ON tabla (col1, col2, ...);

CREATE INDEX idx_nombre ON tabla (col1, col2, ...)
  WHERE predicado;                    -- índice parcial

CREATE INDEX idx_nombre ON tabla
  USING [btree | hash] (col1, col2, ...);
```

Dos tipos principales:

- **B-tree** (el más común, y el default): una estructura jerárquica y ordenada. El tamaño de cada nodo es igual al tamaño de un bloque de disco, por eso entran muchas claves por nodo y el árbol queda "ancho y bajo" — encontrar cualquier fila toma muy pocas lecturas de disco, incluso con millones de filas. Sirve tanto para igualdad (`=`) como para rangos (`>`, `<`, `BETWEEN`) y `ORDER BY`.
- **Hash**: una función hash calcula directamente en qué "cajón" (bucket) está el dato. Es velicísimo para búsquedas por igualdad exacta, pero no sirve para rangos: no puede resolver `>`, `<`, `BETWEEN` ni `ORDER BY`.

Los índices no son gratis. A favor: lecturas mucho más rápidas, menos filas escaneadas por consulta, pueden garantizar unicidad (UNIQUE). En contra: cada INSERT, UPDATE o DELETE debe actualizar también el índice; ocupan espacio en disco; y demasiados índices pueden frenar la escritura sin acelerar las lecturas que realmente importan. Por eso no conviene indexar cada columna "por las dudas": cada índice es una apuesta a cómo se va a consultar la tabla.

7. ¿Cuándo conviene agregar un índice?

Antes de crear uno, cuatro preguntas — juntan varios de los conceptos anteriores en una decisión concreta:

1. **¿Se usa en WHERE, JOIN u ORDER BY?** Si la columna nunca filtra, cruza ni ordena resultados, indexarla no cambia nada.
2. **¿Tiene buena selectividad?** (sección 1) Pocos valores distintos — un booleano, un estado — casi no filtran, aunque tengan índice.
3. **¿La tabla es grande?** En tablas chicas, Postgres prefiere `Seq Scan` aunque exista el índice: ir y volver al índice cuesta más que leer el bloque entero.
4. **¿Qué tipo de índice hace falta?** (sección 6) B-tree si necesitás rangos o ORDER BY; Hash solo si siempre vas a filtrar por igualdad exacta.

Verificación: después de crearlo, confirmalo con `EXPLAIN ANALYZE` (sección 5): si el plan sigue eligiendo `Seq Scan`, el índice no te está ayudando — revisá selectividad y tamaño de tabla antes de conservarlo.

8. Síntesis y cómo seguir estudiando

Cinco ideas para llevarse de toda la clase: el optimizador reescribe tu consulta, no la ejecuta literalmente; filtrar antes y por lo más restrictivo primero es la idea que sostiene casi todo; el álgebra relacional es el lenguaje que usa el optimizador por dentro; EXPLAIN te deja ver ese plan en un motor real; y un índice es lo que hace posible, en la práctica, cada filtro "barato" que vimos.

Bibliografía

- Elmasri, R.; Navathe, S.B. *Fundamentals of Database Systems*. 7ª ed. Pearson (Cap. 17, 18).
- Elmasri, R.; Navathe, S.B. *Sistemas de bases de datos: conceptos fundamentales*. 2ª ed. Addison-Wesley Iberoamericana (Cap. 5 y 16).
- Connolly et al. *Database Systems: A Practical Approach to Design, Implementation and Management*. 2nd ed. Addison-Wesley (Cap. 18).
- Documentación oficial de PostgreSQL — postgresql.org/docs.

Prompt sugerido

Pegá este prompt en la IA junto con el apunte completo (arriba de este bloque):

```
Sos mi tutor para Bases de Datos 2 (Optimización de Consultas e Índices). Te paso
arriba un apunte de la clase: usalo como fuente principal; si algo no está ahí,
decime que no está cubierto antes de inventar una respuesta.

Quiero que:
1. Me tomes preguntas de repaso, una por vez, sección por sección (empezá preguntando
```

por dónde quiero arrancar).
2. Antes de mostrarme una respuesta o un árbol de consulta resuelto, me pidas a mí que lo intente primero.
3. Cuando corrijas, me digas específicamente qué regla o concepto me faltó aplicar (selectividad, cuál regla de oro, IN/EXISTS, B-tree vs Hash, etc.), no solo si está bien o mal.
4. Me generes variantes del ejercicio guiado (otra tabla, otra condición, otro operador) para que practique aplicar las 3 reglas de oro por mi cuenta.
5. Si pido profundizar algo que en el apunte quedó resumido, lo expliques con más detalle, pero conectándolo siempre con los términos que ya usa el apunte (σ, π, ⋈, ⋉, selectividad) para no mezclar terminología distinta.